CONSTRAINT-BASED LOGIC QUESTIONS (Design Your Own Condition)

These questions don't just ask for a program — they impose a restriction that forces students to think about *how* operators actually work, not just plug in a formula.

E1. Number Classification Given an integer `x`, classify it as one of:

- Positive even
- Positive odd
- Negative even
- Negative odd
- Zero

Use a combination of relational, logical, and bitwise operators. **Restriction:** Do not use `%`. *(Hint: think about what the last bit of a number tells you, and how bitwise `&` can extract it.)*

E2. Square Comparison with Special Case For an integer `x`, determine which of the following is true:

- `x² > 100`
- `x² == 100`
- `x² < 100`

But if `x` is negative, classify it separately as `"Negative input"` **before** any comparison is made. Write the decision logic using only arithmetic and relational operators (no `abs()`, no `sqrt()`).

E3. Debugging Challenge — Operator Precedence Trap Explain the difference between:

```
x*x > 100
```

and

```
x*(x > 100)
```

For `x = 12`, evaluate both expressions step by step and state the final value each one produces. Then explain, in your own words, why the second expression is (almost) never what a programmer actually intends.

E4. Sign & Parity Without Comparison Operators Given an integer `x`, determine whether it is even/odd **and** whether it is positive/negative/zero — but you are **not allowed to use `if`, `%`, or `</>`** anywhere. You may only use bitwise operators (`&`, `|`, `^`, `>>`), arithmetic operators, and the ternary operator `?:`. *(This forces students to think about sign bits and the least significant bit directly.)*

E5. Range Classification Without `&&` Given an integer `marks` (0–100), classify it into `Fail (<40)`, `Pass (40–59)`, `Good (60–79)`, `Excellent (80–100)` — but you are **not allowed to use the `&&` operator**. Use only nested `if` or `else if` chaining to achieve the same effect. *(Purpose: shows that `&&` is convenience, not necessity — nesting/ordering can replace it.)*

E6. Predict Before You Run For `x = 5`, predict the output of each line **before** compiling, then explain any surprising result:

```
printf("%d\n", x & 1);      // bitwise AND with 1
printf("%d\n", x % 2);      // modulus with 2
printf("%d\n", x >> 1);     // right shift by 1
printf("%d\n", x / 2);      // integer division by 2
```

Which pairs give identical results for **positive** `x`, and would that still hold true if `x` were **negative**? Why or why not?

PLACEMENT-ORIENTED HOTS — APPLIED LOGIC BUILDING

These are written the way logic-building rounds in placement tests/interviews are framed: a real scenario or constraint, not a "define X" question. All of them are solvable using **only** what's been taught so far — variables, operators, `if / else if / else` — no loops, arrays, or functions, so students are forced to build the logic purely through conditions and operator tricks. This is exactly the kind of thinking interviewers probe for.

F1. Swap Without a Temporary Variable Write a program to swap two integers `a` and `b` **without using a third variable**. Do it in two different ways: (i) using arithmetic operators (`+`, `-`), (ii) using the bitwise XOR operator (`^`). Then explain: what happens if both methods are attempted when `a` and `b` are stored at the **same memory address** (i.e., you accidentally do `a = a ^ a`)? Trace it and explain why this is a real bug seen in production code.

F2. Largest of Three — Without Using `if-else` Chains Find the maximum of three integers `a`, `b`, `c` using **only the ternary operator**, in a single line/expression (no `if` statements at all). Follow-up: Rewrite it so it also handles the case where two or more numbers are equal to the maximum, printing `"Tie"` instead.

F3. Quadrant Finder Given a point `(x, y)` on the Cartesian plane, write a program to determine:

- Which quadrant it lies in (I, II, III, IV)
- Or if it lies **on the X-axis, Y-axis, or at the Origin**

This requires careful ordering of conditions — get the order wrong and the origin gets misclassified as being "on the X-axis." Explain why the order of your `if` conditions matters here.

F4. Triangle Validity + Type — In One Program Given three sides `a`, `b`, `c`, write a single program that:

1. First checks if a valid triangle can be formed (sum of any two sides must be greater than the third)
2. If valid, further classifies it as Equilateral, Isosceles, or Scalene
3. If invalid, prints `"Not a triangle"` and skips classification entirely

(Placement angle: interviewers specifically check whether you validate before classifying — a very common mistake is to classify first and validate as an afterthought.)

F5. Set-Bit Style Logic (No Loops) An integer `n` is guaranteed to be between 0 and 15 (fits in 4 bits). Without using loops or arrays, write a program using only `if-else` and bitwise `&` to count how many bits are set to `1` in `n`. *(Hint: you only have 4 bit positions to check — you can check each one directly.)*

F6. Opposite Signs Check — The Interview Classic Write a program to check whether two integers `a` and `b` have opposite signs — **without using multiplication, division, or any relational operator (<, >)**. (Bitwise XOR is your only tool here.) Explain *why* `(a ^ b) < 0` correctly detects opposite signs by reasoning about the sign bit. *(Note: technically this uses one relational operator for the final check — ask students if they can spot that the restriction is about how the sign detection itself is computed, not the final comparison.)*

F7. Single-Digit FizzBuzz Logic (No Loop Version) Take a single integer `x`. Print `"Fizz"` if divisible by 3, `"Buzz"` if divisible by 5, `"FizzBuzz"` if divisible by both, and the number itself otherwise. This is a one-shot (no-loop) version of the classic FizzBuzz interview question — get the **order of your `if` conditions** right, or your program will never print `"FizzBuzz"` at all. Explain why.

F8. ASCII-Only Character Classifier Given a single character input, classify it as Uppercase Letter, Lowercase Letter, Digit, or Special Character — using **only relational operators on the ASCII value** (no `isalpha()`, `isdigit()`, etc. from `<ctype.h>`). Follow-up: extend it to also convert uppercase to lowercase (and vice versa) using arithmetic on the ASCII value, without any built-in function.

F9. Absolute Value Without `abs()` Write a program to find the absolute value of an integer `x` **without using the `abs()` function and without using an `if` statement** — using only arithmetic and the ternary operator, or a bitwise trick involving right-shift and XOR. *(Interviewers use this to test whether you understand two's complement representation, not just syntax.)*

F10. Debugging — The Silent Precedence Bug A student wants to check if `x` is odd using a bitwise trick and writes:

```
if (x & 1 == 1)
    printf("Odd");
else
    printf("Even");
```

This *appears* to work correctly for most test cases — which is exactly what makes it dangerous. Explain step by step (using C's operator precedence table) what this condition is **actually** evaluating, why it happens to give the right answer for `1` and `0`, and rewrite it correctly with proper parentheses. Then construct a value of `x` for which the *unparenthesized* version would silently fail even at the boolean-result level, to prove the bug is real and not just cosmetic.

F11. Divisibility by 4 Without `%` Write a program to check whether a number is divisible by 4 using only bitwise operators (hint: check the last 2 bits). Generalize your explanation: how would the same trick work for checking divisibility by 8, or by any power of 2?

F12. Predict, Then Justify — Mixed Operator Chain Without compiling, predict the output, then verify by running it:

```
int a = 5, b = 10, c = 15;
int result = (a < b) && (b < c) || (a > c);
printf("%d", result);

int x = 4, y = 4;
printf("%d", x == y == 1);
```

For the second line specifically: does it print what students *expect*? Explain how `==` being left-associative changes the meaning of `x == y == 1` compared to what a student might assume it means ("check if x equals y equals 1").

BOOLEAN LOGIC — CATEGORIZATION & CLASSIFICATION PROBLEMS

This section groups classic "classify the input into a category" problems by theme, each taken through three difficulty levels — so students see the *same real-world logic pattern* get progressively harder, rather than jumping between unrelated topics. This mirrors how classification/eligibility logic actually shows up in placement coding rounds.

G1. Odd/Even & Positive/Negative/Zero

G1.1 (Easy) Write a program to check whether a number entered by the user is odd or even using the modulus operator.

G1.2 (Intermediate) Write a **single** program that classifies an integer into exactly one of five categories — Positive Even, Positive Odd, Negative Even, Negative Odd, or Zero — using nested `if` and logical operators.

G1.3 (HOTS) Extend the classifier above to also correctly handle `INT_MIN` (the smallest possible `int` value, e.g. `-2147483648`). If your classification logic ever computes `-x` (the negation) to check something, explain why this specific value can silently produce a **wrong answer or undefined behavior**, and how you would guard against it. *(This is a genuine, well-known interview gotcha about integer range and two's complement — not just a theoretical edge case.)*

G2. Vowel / Consonant

G2.1 (Easy) Write a program to read a single lowercase character and determine if it is a vowel or a consonant using `if-else`.

G2.2 (Intermediate) Extend the program to: (i) work for **both uppercase and lowercase** input, and (ii) first validate that the input is actually an alphabet at all — printing `"Invalid input"` for digits or symbols before even checking vowel/consonant.

G2.3 (HOTS) Rewrite the vowel check as a **single compound boolean expression** (one `if` condition, no chained `else if` for each vowel). Then explain a neat trick: converting any uppercase letter to lowercase using `ch | 0x20` (bitwise OR) instead of `ch + 32` or `tolower()`. Why does `| 0x20` work for letters specifically, and why would it silently give a **wrong** result if accidentally applied to a digit or symbol character?

G3. Leap Year Checking

G3.1 (Easy) Write a program to check if a year is divisible by 4 (a simplified, *incomplete* leap year check).

G3.2 (Intermediate) Write the **complete and correct** leap year check: divisible by 4, but not by 100 — unless it is also divisible by 400.

G3.3 (HOTS) Given a full date as three integers `day`, `month`, `year`, write a program to validate whether that date is a **legal calendar date** — specifically, correctly reject `31-04-2024` (April has only 30 days) and correctly accept `29-02-2024` but reject `29-02-2023`. This forces you to combine the leap year check with a month-based day-limit check in one connected piece of logic (nested `if` / `switch` for days-per-month, feeding into the leap year condition for February specifically).

G4. Alphabet, Digit, or Special Symbol

G4.1 (Easy) Write a program to check whether an entered character is an alphabet or a digit using ASCII range checks (no `<ctype.h>` functions).

G4.2 (Intermediate) Extend it to a full **4-way classification**: Uppercase Letter, Lowercase Letter, Digit, or Special Character — purely from ASCII value ranges.

G4.3 (HOTS) Write a **single-character "password strength" checker**: given one character at a time, classify it as contributing to "Has Uppercase", "Has Lowercase", "Has Digit", or "Has

Special Symbol". Then explain conceptually (in comments or a short paragraph) how this single-character classifier would be **reused four times in a row** to validate a full password like `"Abc123!"` requires at least one of each category — even though you haven't learned loops/arrays yet, describe the logic in words as if you were explaining it to design the full version later.

G5. Eligibility Checks (Loan / Scholarship)

G5.1 (Easy) Write a program to check loan eligibility based on a single combined condition: the applicant is eligible only if `age >= 21` **and** `monthly_income >= 25000`. Print `"Eligible"` or `"Not Eligible"` accordingly.

G5.2 (Intermediate) Write a scholarship eligibility program with **two independent paths to qualify**:

- Merit-based: `percentage >= 90` (qualifies regardless of family income), **or**
- Need-based: `percentage >= 75` **and** `family_income <= 200000`

Use logical `&&` and `||` together correctly (with parentheses to make precedence explicit) to combine both paths into one condition.

G5.3 (HOTS) Design a **loan approval classifier** with three possible outcomes — `"Approved"`, `"Approved with higher interest rate"`, or `"Rejected"` — based on combining `credit_score`, `monthly_income`, and `existing_loans_count` using multiple conditions. Then answer as a mini debugging exercise: a colleague's implementation accidentally used `||` where it should have used `&&` when combining the credit-score and income conditions. Explain, with a concrete example of applicant numbers, how this single operator mistake could cause the system to **wrongly approve a genuinely ineligible applicant** — and why this class of bug is especially dangerous precisely because it doesn't cause a crash or compiler warning, just a silently wrong business decision.